

Personal Expense Tracker with Smart Budget Alerts

COVER PAGE

Project Title: Personal Expense Tracker with Smart Budget Alerts

Name: Animesh Singh

Registration Number: 25BCE11355

Course: Programming in Java

Institution: VIT Bhopal University

Branch: B.Tech Computer Science and Engineering (Core)

Submission Date: September 2026

GitHub Repository URL: [Insert Repository Link]

INTRODUCTION

This project implements a command-line personal finance tracker in Java. It allows a user to record income and expense transactions, organize them by category, persist them between sessions, and generate spending reports. Its distinguishing feature is a concurrent monitoring component that evaluates spending against per-category budget limits on a background thread and issues warnings in real time rather than only on request.

The project was chosen to exercise the core areas of the course syllabus in a single coherent application: exception handling, multithreading and synchronization, the Collections Framework, String and array manipulation, and character-oriented I/O streams.

As a first-year student living in the university hostel, managing a monthly allowance efficiently—balancing cafeteria meals, shared room supplies, and personal expenses—has been a practical challenge. I chose this idea over standard library or inventory management systems because I wanted a tool that didn't just passively record spending but actively warned me when I was nearing my limits. It allowed me to apply the core concepts of Java, specifically multithreading and data persistence, to a problem I actually face every day.

PROBLEM STATEMENT

Expense tracking tools are typically passive—they act as digital ledgers that record spending but surface the consequences only when the user proactively checks their balance or generates a report. In many cases, this feedback arrives after the purchasing decision has already been made, defeating the purpose of a budget.

This project addresses that gap by making the monitoring continuous and concurrent with the application's use. By running a background monitor

alongside the main interface, the system can alert the user the moment a specific category's budget threshold is crossed, allowing for immediate course correction.

FUNCTIONAL REQUIREMENTS

Requirement	Description	Module
1	Add income and expense transactions with amount, category, type, description, and date	Main, ExpenseTracker
2	View all transactions in a formatted listing	Main, ExpenseTracker
3	Delete a transaction by its unique identifier	ExpenseTracker
4	Filter transactions by category and by type	ExpenseTracker
5	Validate all user input and reject invalid entries with a clear message	Main, ExpenseTracker.validate()
6	Persist transactions to a CSV file and reload them on startup	FileManager

Requirement	Description	Module
7	Monitor category spending continuously in the background	BudgetMonitor
8	Emit a warning at 80% and an alert at 100% of a category budget	BudgetMonitor
9	Generate a summary report with totals and per-category spend against budget	ReportGenerator
10	Produce a category-by-month spending grid	ReportGenerator
11	Write the generated report to a text file	FileManager.writeReport()

NON-FUNCTIONAL REQUIREMENTS

Requirement	How it is met
1: Reliability	The application must not terminate on bad input or corrupt data. Custom checked exceptions (InvalidTransactionException, FileParseException) are implemented; malformed CSV lines are skipped individually rather than aborting the load; input helpers re-prompt on failure.

Requirement	How it is met
2: Error Handling	Deliberate checked/unchecked split. Recoverable input and parse errors are checked and must be handled; budget breaches are unchecked notifications. Multi-catch is used where several exception types represent one logical failure.
3: Concurrency Safety	Shared state remains consistent under concurrent access. All public ExpenseTracker methods are synchronized; BudgetMonitor.running is volatile; getAllTransactions() returns a defensive copy.
4: Usability	Operable with no prior knowledge. Uses numbered menus, numeric selection for categories, blank date defaults to today, and requires no setup or config files.
5: Maintainability	Modular structure with clear separation of concerns across presentation, logic, model, and persistence layers. No single class exceeds ~130 lines.
6: Resource Efficiency	BufferedReader reads line-by-line rather than loading the whole file into memory. Try-with-resources guarantees stream closure. The monitor thread sleeps between cycles instead of busy-waiting.

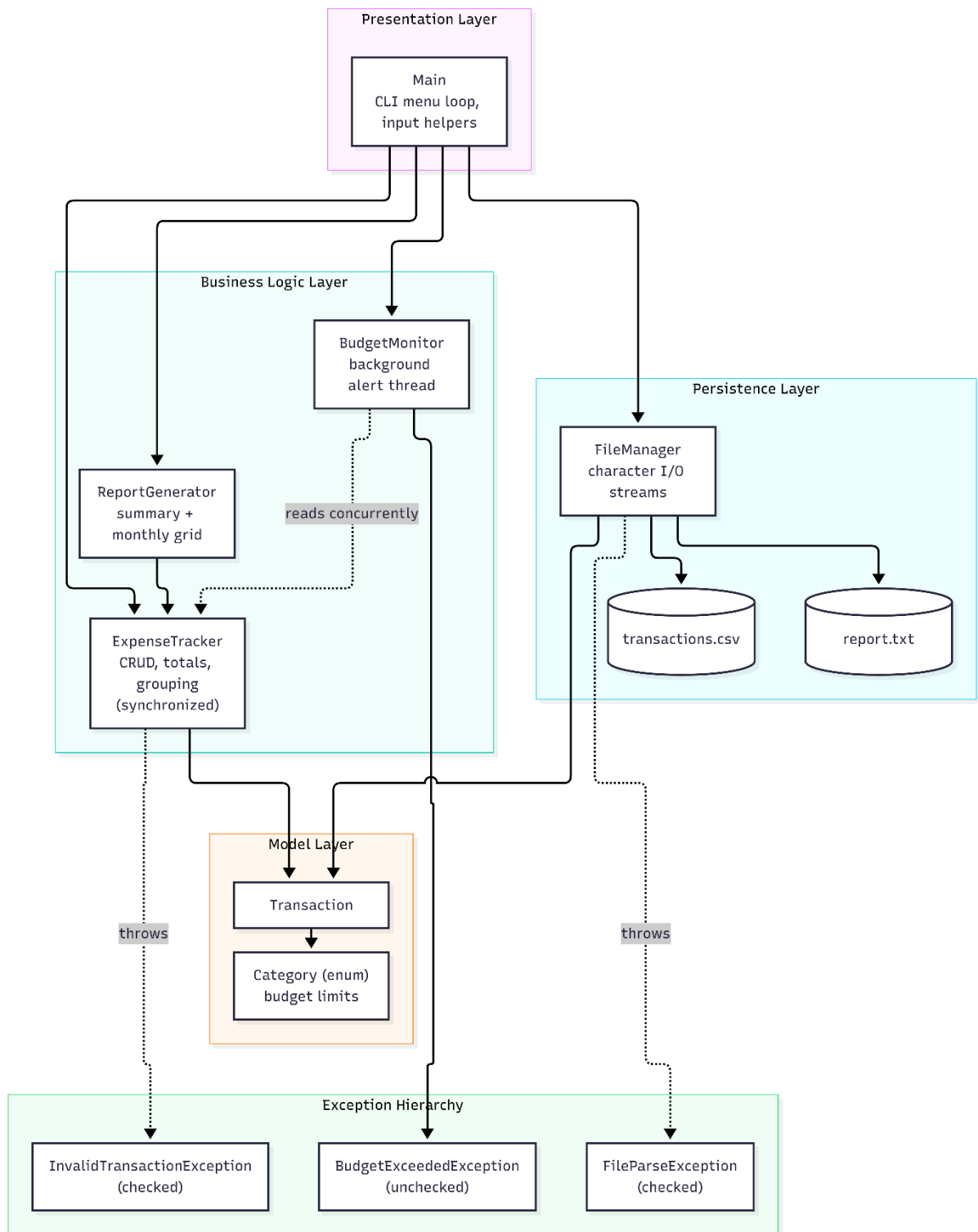
SYSTEM ARCHITECTURE

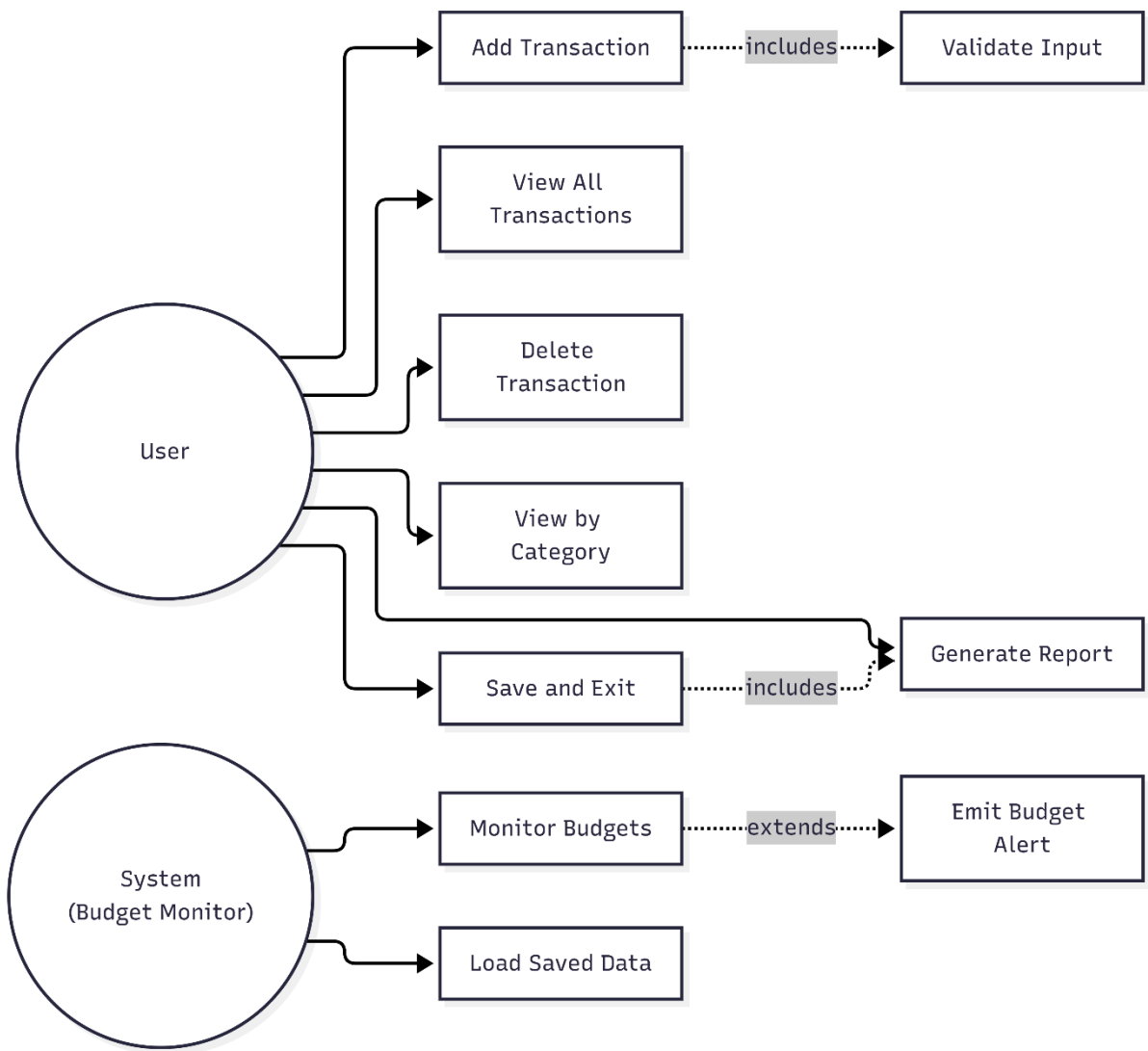
The application is structured into four distinct layers to ensure a clean separation of concerns:

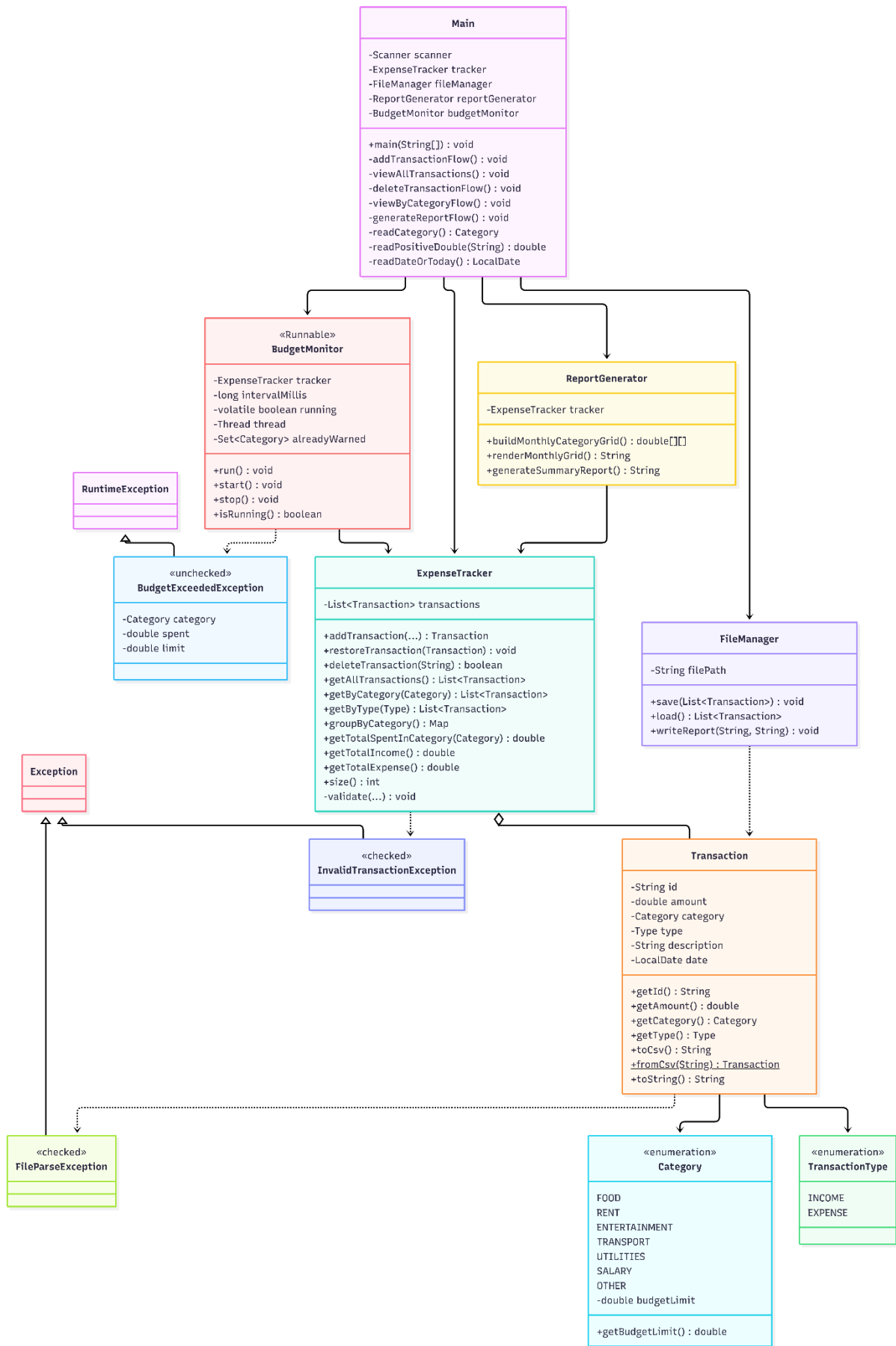
1. **Presentation Layer (Main):** Manages the menu loop, collects input, and handles initial validation. It contains no business rules.
2. **Business Logic Layer (ExpenseTracker, ReportGenerator, BudgetMonitor):** Handles calculations, filtering, and monitoring. ExpenseTracker owns the authoritative in-memory transaction list.
3. **Model Layer (Transaction, Category):** Defines the data representation and serialization formats.
4. **Persistence Layer (FileManager):** The only component that interacts with the filesystem for data storage and retrieval.

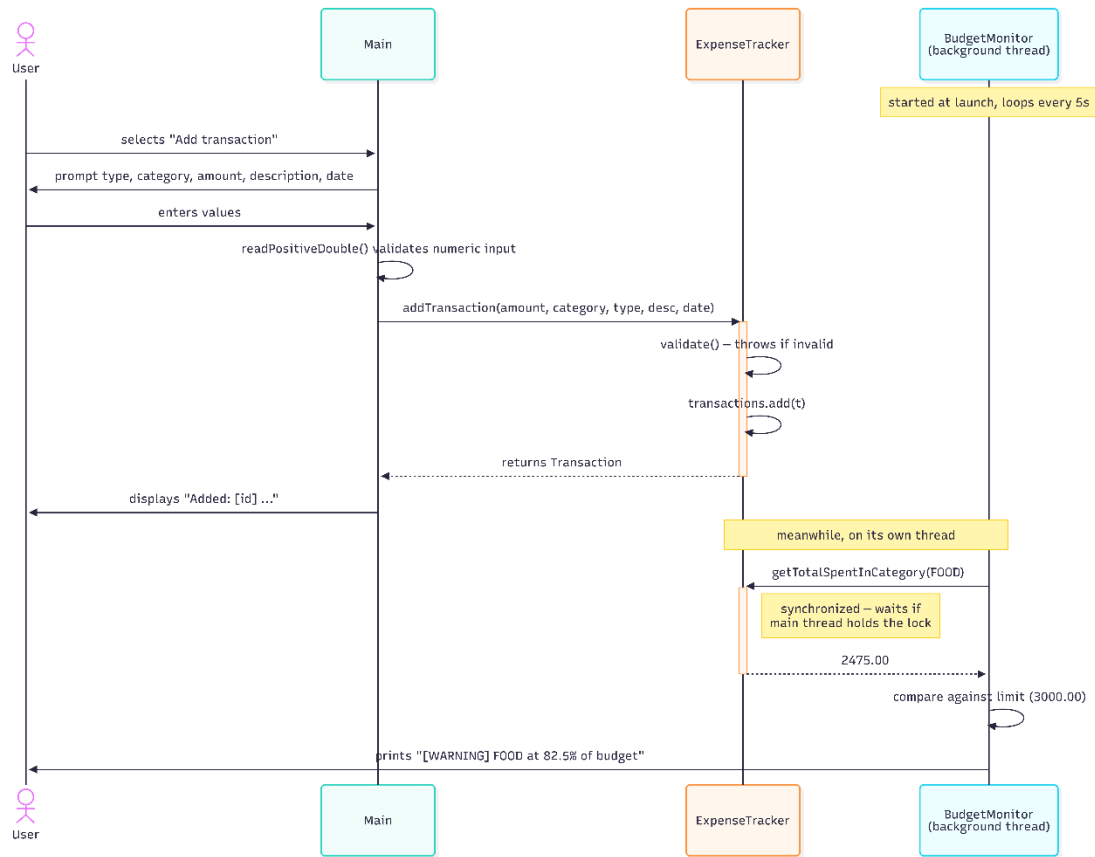
Concurrency Boundary: The BudgetMonitor runs on a separate thread and is the only component that reads ExpenseTracker while the main thread may be actively writing to it. Because of this, synchronization is handled within ExpenseTracker rather than the monitor, ensuring the state remains locked during mutations.

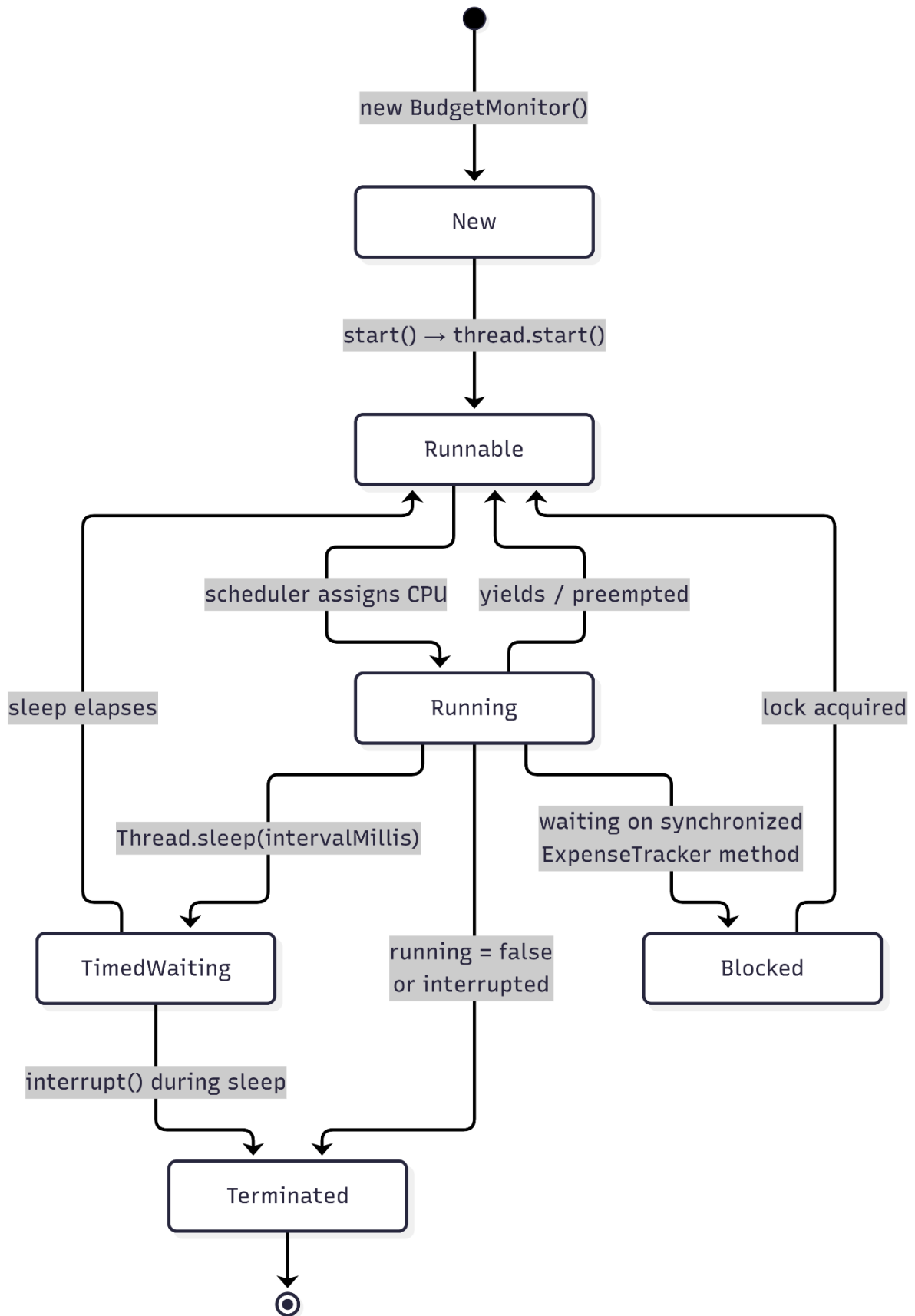
DESIGN DIAGRAMS







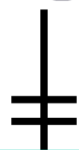




TRANSACTION		
string	id	PK
date	date	
enum	type	
enum	category	FK
double	amount	
string	description	



belongs to



CATEGORY		
string	name	PK
double	budgetLimit	

DESIGN DECISIONS AND RATIONALE

Enum for Category rather than String: I utilized an Enum for transaction categories to enforce strict type safety and tie budget limits directly to each constant. The downside I noticed is that adding a new category requires altering the source code and recompiling, rather than letting the user add one dynamically. However, for a rigid personal budget, this design prevents accidental typos (like typing "Fod" instead of "Food") from fragmenting the data.

Checked vs Unchecked Exceptions: I deliberately split my exception hierarchy. `InvalidTransactionException` and `FileParseException` are checked because the system can logically recover from them by re-prompting the user or skipping a file line. Conversely, `BudgetExceededException` is unchecked because it acts as a notification. I initially tried making budget breaches checked exceptions, but it forced me to wrap basic transaction additions in cumbersome try-catch blocks, which cluttered the core logic.

Synchronizing ExpenseTracker over the Monitor: The shared mutable state (the transaction list) lives inside the `ExpenseTracker`. Synchronizing the monitor itself wouldn't prevent the main thread from modifying the list during a read. Locking the tracker ensures thread safety at the source of truth.

volatile on the running flag: Without marking the running flag as volatile, the background thread might cache the value in its local memory and ignore the stop signal sent by the main thread during shutdown.

Flat CSV over a Database: I opted for a flat CSV file rather than a database like SQLite. This kept the project strictly focused on mastering Java's core I/O streams and Collections rather than wrestling with JDBC configurations. It is also entirely portable and human-readable for manual debugging.

Skipping Malformed Lines: During file loading, if a single CSV row is corrupted, throwing a fatal error would lock the user out of their entire

history. I chose to log the error and skip the line, treating persistence as highly fault-tolerant.

Defensive Copying: `getAllTransactions()` returns a copy of the list, preventing any external code from accidentally bypassing the synchronization lock and mutating the internal state.

IMPLEMENTATION DETAILS

Exception Handling

- Created three custom exception classes (two checked, one unchecked).
- Used throw in `ExpenseTracker.validate()` and `Transaction.fromCsv()`.
- Propagated exceptions using throws through the call chain from `addTransaction()` up to Main.
- Utilized multi-catch in `Transaction.fromCsv()` to handle both `IllegalArgumentException` and `DateTimeParseException` cleanly.
- Employed try-with-resources in `FileManager` to guarantee stream closure.

Multithreading

- `BudgetMonitor` implements `Runnable` and is executed on a dedicated Thread.
- Thread is set as a daemon so it doesn't block JVM shutdown.
- Lifecycle management via `start()`, `run()` (loops while running), and `stop()` (clears flag and interrupts).
- Uses `Thread.sleep()` between evaluation cycles to save CPU cycles.
- Cross-thread visibility is guaranteed using a volatile running flag and synchronized methods on the shared `ExpenseTracker`.

Collections Framework

- `ArrayList<Transaction>` serves as the primary data store.

- `HashMap<Category, List<Transaction>>` is used for category grouping, heavily utilizing `computeIfAbsent`.
- `EnumSet<Category>` within the monitor prevents redundant alert spamming.

Strings and Arrays

- Extensive use of `String.split()`, `String.join()`, `isBlank()`, and `replace()` for CSV parsing.
- `String.format()` and `printf` used for clean, aligned console UI.
- Utilized a 2-D array `double[][]` in `ReportGenerator.buildMonthlyCategoryGrid()` to map out the category-by-month spending matrix.

I/O Streams

- Used `BufferedReader` and `BufferedWriter` over `FileReader/FileWriter` for efficient line-by-line character input/output.
- Checked `File.exists()` to handle the first-run scenario gracefully when no CSV exists yet.

SCREENSHOTS AND RESULTS

Compilation: Clean build via `javac *.java` with no errors.

Main Menu: Startup screen displaying the loaded transaction count.

Valid Transaction: Successfully adding a standard expense.

Input Validation: Rejecting a negative amount/string input with a clear error message.

Concurrency in Action: A background [WARNING] threshold alert appearing mid-session while the menu is actively waiting for input.

Alert Escalation: The warning upgrading to an [ALERT] upon breaching 100% of the category limit.

Report Generation: The formatted output of the summary report.

Data Persistence: The raw transactions.csv viewed in a text editor.

Fault Tolerance: Console successfully skipping a deliberately corrupted line in the CSV.

```
=== Personal Expense Tracker ===

1. Add transaction
2. View all transactions
3. Delete a transaction
4. View transactions by category
5. Generate report
6. Save & Exit
Choose an option:
1
Type - 1) Income 2) Expense: 2
Category:
  1) FOOD
  2) RENT
  3) ENTERTAINMENT
  4) TRANSPORT
  5) UTILITIES
  6) SALARY
  7) OTHER
Choose: 1
Amount: 12345
Description: Food
Date (YYYY-MM-DD, or leave blank for today):
Added: [691f713e] 2026-09-17 | EXPENSE | FOOD | 12345.00 | Food

1. Add transaction
2. View all transactions
3. Delete a transaction
4. View transactions by category
5. Generate report
6. Save & Exit
Choose an option:

[ALERT] Budget EXCEEDED for FOOD: spent 12345.00 / limit 3000.00 (412%)

[WARNING] FOOD is at 412% of its budget (12345.00 / 3000.00)
```

TESTING APPROACH

#	Scenario	Input	Expected Output	Actual Result	Pass/Fail
1	Reject negative amount	-100	Caught, prompt user for valid amount	Re-prompted successfully	Pass
2	Reject non-numeric amount	abc	Caught, prompt user for valid amount	Re-prompted (Added InputMismatchException handling to fix initial crash)	Pass
3	Invalid date fallback	31-02-2026	Defaults to today's date	Defaulted to current date	Pass
4	Persistence round-trip	Add 3 items, exit, relaunch	All 3 transactions reloaded	Data loaded seamlessly	Pass
5	Corrupt line recovery	Edit CSV: this,is,broken	Line skipped, remainder loaded	Printed skip warning, loaded rest	Pass

#	Scenario	Input	Expected Output	Actual Result	Pass/Fail
6	80% threshold warning	FOOD expense > ₹2400	[WARNING] printed within 5s	Thread output warning	Pass
7	100% threshold alert	FOOD expense > ₹3000	[ALERT] printed within 5s	Thread output alert	Pass
8	Alert not repeated	Continue after alert	No duplicate alert spam	Alert fired only once per threshold	Pass
9	Clean shutdown	Save & Exit	Thread stops, data saved to CSV	Application exited gracefully	Pass
10	Invalid menu option	99	"Invalid option" message	Menu looped securely	Pass

CHALLENGES FACED

Filename Casing Errors Early in development, I ran into a frustrating compilation failure where Java filenames did not perfectly match the public class names. Coming from environments where casing can sometimes be forgiving, I was hit with dozens of cascading "cannot find symbol" errors. Tracing these back to a simple uppercase/lowercase mismatch took time but reinforced how strict Java's compiler is regarding file organization.

Reasoning About Concurrency Figuring out exactly how the BudgetMonitor thread could read the totals while the Main thread was simultaneously adding new transactions was confusing. Initially, I wondered if synchronization was overkill. However, mapping it out logically made me realize that without locking the state, the monitor could read a partially updated transaction list, resulting in missed budget warnings. This was my "aha" moment regarding why locking shared mutable state is non-negotiable.

Handling Corrupt Data States Deciding how the system should react to a corrupted CSV line was a practical design challenge. My first instinct was to throw a critical error if the file parse failed. But realizing that one badly formatted row could lock a user out of hundreds of valid transactions changed my approach. I had to design a recovery flow that catches the error, logs a warning to the console, and gracefully continues processing the next line.

LEARNINGS AND KEY TAKEAWAYS

Implementing the background thread taught me that reading about Runnable is entirely different from actually managing a thread's lifecycle. Designing the system showed me that the real complexity of concurrency isn't starting the thread—it's safely sharing memory between threads. Understanding how the volatile keyword ensures memory visibility across threads was a major breakthrough.

Designing my own exception hierarchy fundamentally changed how I view error handling. Rather than just using try-catch blocks to prevent the program from crashing, I started using exceptions as a structured way to communicate state between the architecture's layers, making deliberate

choices about what callers *must* handle versus what they should just be aware of.

Finally, integrating the Collections framework—especially utilizing HashMap to group transactions by category—felt incredibly practical. It replaced what would have been pages of complex iterative array logic with a few lines of clean, readable code.

FUTURE ENHANCEMENTS

Dynamic Configuration: Loading category budget limits from a configuration file rather than hardcoding them into the Category enum.

Advanced Analytics: Implementing date-range filtering and month-over-month comparisons.

Automation: Allowing for recurring transactions (e.g., rent, subscriptions) to apply automatically based on the system clock.

Data Export: Adding functionality to export the generated reports cleanly to PDF.

Database Integration: Migrating from flat-file CSV storage to an embedded database such as SQLite.

Concurrency Upgrades: Replacing the raw Thread implementation with a ScheduledExecutorService for more robust background task management.

Automated Testing: Writing formal unit tests using JUnit for ExpenseTracker logic and Transaction.fromCsv() parsing.

REFERENCES

☞Oracle Java SE Documentation — `java.util.concurrent`, `java.io`, `java.time`

Oracle Java Tutorials — Concurrency, Exceptions, and Collections Frameworks

Course Materials: Programming in Java (VIT Bhopal University)

Mermaid.js Documentation for generating architecture and sequence diagrams